

No.	Question	Interview-Ready Answer
20	Re-renders & per A component re-renders when its state changes or when its parent component re-renders. Key optimization techniques include: using React.memo (for components), useMemo/useCallback (for values/functions), Key prop stability, and using Virtualization (like react-window) for large lists.	
21	Reconciliation	Reconciliation is the process of updating the browser DOM to match the latest React State. React performs this by comparing the new Virtual DOM tree with the old one, identifying differences (Diffing), and applying the minimal necessary changes to the real DOM in a single efficient batch.
22	Diffing algorithm	The Diffing algorithm is the heuristic set of rules used during Reconciliation to calculate changes. It is optimized for speed, not perfection, based on two main assumptions: 1) Two elements of different types will produce different trees, and 2) Developers use a stable Key prop to denote which children are stable across renders.
23	React.lazy & Suspense	React.lazy is used for Code Splitting—it allows you to load a component's code only when it's needed (on-demand). Suspense is a component that wraps the lazy-loaded component and allows you to display a fallback UI (like a spinner) while the component's code is being fetched and loaded over the network.
24	Error boundaries	An Error Boundary is a class component that implements either static getDerivedStateFromError() or componentDidCatch(). It is used to catch JavaScript errors that occur in its child component tree, log the error, and render a fallback UI instead of crashing the entire application.
25	Auth & protected View	We typically handle this using React Router and a Wrapper Component (or custom layout). The wrapper component checks the user's authentication status (from a Context/Redux store). If authenticated, it renders the protected component; if not, it redirects the user to the login page via the Maps function.
26	Render props vs Both patterns share logic, but Render Props (passing a function as a prop to control rendering) offer more explicit control over the final output and are less prone to naming collisions than HOCs (which are component wrappers). Custom Hooks are the modern preferred pattern for logic reuse over both.	
27	SSR vs CSR	SSR (Server-Side Rendering) generates the HTML on the server before sending it to the client. It provides a faster First Contentful Paint (FCP) and better SEO. CSR (Client-Side Rendering) sends a minimal HTML shell, and the browser runs JavaScript to render the UI. It provides a faster transition between views but poorer initial load performance and SEO.
28	React Fiber & Co	React Fiber is the complete re-implementation of React's core Reconciliation algorithm. It allows rendering work to be pausable, resumable, and interruptible. Concurrent Mode (React 18+) leverages Fiber to enable React to work on multiple tasks simultaneously (like transitioning the UI while fetching data), significantly improving the perceived responsiveness of the application.
29	Testing React	We primarily use Jest for the test runner and React Testing Library (RTL) for writing tests. RTL focuses on testing components from the user's perspective (e.g., clicking a button, verifying the result) rather than testing implementation details (like internal state), promoting robust and maintainable tests.

No.	Question	Interview-Ready Answer
11	React Hooks: useState is the hook that allows functional components to manage local state. useEffect handles side effects—tasks that interact with the outside world, like data fetching, DOM manipulation, timers, or subscriptions. Its dependency array controls when the effect runs (empty array [] = mount only; with dependencies = update when dependencies change).	
12	Controlled vs Uncontrolled components have their form data (e.g., input value) handled entirely by React state. This is the recommended approach, as it gives you immediate control over validation and data flow. Uncontrolled components store their data internally in the DOM, and you access it using a Ref when needed.	
13	React Router & Context Router is the standard library for client-side routing. It allows the browser URL to change without requesting a new page from the server, improving the user experience. It uses the History API in the browser to manage navigation and render specific components based on the route path.	
14	Context API vs Redux: Context API is a built-in solution for light-to-moderate global state that avoids prop drilling. It is simple but can cause performance issues if used for high-frequency updates, as any consumer re-renders. Redux (or similar libs like Zustand) is used for complex, large-scale state management. It is more centralized, provides excellent debugging tools, and offers better performance optimization via selectors.	
15	Prop drilling & so Prop drilling is the practice of passing props down through multiple layers of components that don't need the data, simply to reach a deeply nested child component. The main solutions are the Context API (for local application state) or a global state management library (like Redux or Zustand).	
16	React.memo is a Higher-Order Component (HOC) used to optimize functional components. It memoizes the component and only re-renders it if its props have shallowly changed. This is crucial for preventing unnecessary re-renders of heavy or complex components whose props are static.	
17	useMemo vs useMemo caches the result of a function (a value) and is used to prevent expensive calculations from running on every render. useCallback caches the function definition itself and is used to prevent a function reference from changing, which is important when passing functions as props to components wrapped in React.memo.	
18	Higher-Order Coi An HOC is a function that takes a component and returns a new component with enhanced props or logic. They are used for reusing component logic, such as authentication checks (withAuth) or connecting to a data source. They are being gradually replaced by Custom Hooks in modern React.	
19	Handling forms & Forms are best handled using controlled inputs. This means you must: 1) Bind the input's value prop to a piece of state (useState), and 2) Update that state via an onChange handler for every keystroke. This guarantees that React state is the single source of truth for all form data.	

No.	Question	Interview-Ready Answer
1	What is React?	React is a JavaScript library for building user interfaces, focused on the Component-Based Architecture (CBA). It works by maintaining a Virtual DOM, which allows it to efficiently update the actual browser DOM only when necessary, leading to high performance.
2	Functional vs Class Components	Functional Components are JavaScript functions that return JSX. They are the modern standard, primarily using Hooks to manage state and side effects. Class Components are ES6 classes that extend React.Component and use lifecycle methods, but are generally considered legacy.
3	Props vs State	Props (short for properties) are used to pass data down from a parent component to a child component. They are immutable (read-only). State is data that is managed within the component, used for internal logic, and is mutable (can be changed via useState).
4	What is JSX?	JSX (JavaScript XML) is a syntax extension for JavaScript that allows us to write HTML-like code within our JavaScript files. It is not mandatory, but it makes component structure intuitive. It ultimately compiles down to React.createElement() calls.
5	Creating a simple functional component	A simple functional component is a plain JavaScript function that accepts props object as an argument and returns a piece of UI defined in JSX.
6	Virtual DOM	The Virtual DOM (VDOM) is a lightweight copy of the actual browser DOM. When data changes, React creates a new VDOM tree and uses a Diffing Algorithm to compare it to the previous tree, calculating the minimal changes needed. It then updates only those specific changes in the real DOM, making the update process fast.
7	Key prop in lists	The key prop is a stable, unique identifier that React requires for every item in a list. It helps React's Reconciliation process efficiently track which items have been added, removed, or reordered. Using the index as a key is an anti-pattern if the list items can be reordered, as it leads to bugs and performance issues.
8	Event handling	React uses a Synthetic Event System, which is a wrapper around the browser's native event system. Events are named using camelCase (e.g., onClick), and the event handlers are passed functions, not strings. This ensures cross-browser compatibility.
9	Default props	defaultProps provide a fallback value for props if the parent component doesn't pass them. In modern React, it's often simpler to use ES6 default function parameters directly in the function signature for functional components.
10	Conditional rendering	This is the process of deciding which UI elements to render based on the current state or props. Common techniques include the JavaScript ternary operator (condition ? A : B), the Logical && operator (condition && <Component />), and standard if/else statements within the function body.